

Lab Work 08

Topic	Using AI for scaffolding, method implementation, and debugging OOP code.
Objective	Leverage an AI assistant to accelerate code writing and identify/fix bugs in the OOP context.
AI Copilot Usage	Allowed and encouraged

Important Notes

- This is an individual task. All code must be your own original work.
- Ask questions and seek help from the lecturer or teaching assistants if you are stuck.
- Focus on writing clean, well-organized, and well-documented code.

Submission

- Submit updated ***school_simulator.py*** (with partially implemented ***EnrollmentSystem***) and your prompt history for this session.

Tasks

1. Scaffolding a Class:

- Prompt:** Ask your AI assistant: "Generate a Python class scaffold for ***EnrollmentSystem***. It should follow the Singleton pattern. Its ***__init__*** method should get the instance of ***SchoolRegistry*** (assume ***SchoolRegistry*** is a Singleton class that exists). Define empty methods ***enroll_student_in_unit(self, student_id, unit_code)***, ***get_unit_enrollments(self, unit_code)***, and ***get_student_schedule(self, student_id)*** with basic docstrings."
- Review the generated scaffold. Copy it into your file. Does the Singleton look right? Are methods defined? (You'll likely need to add logic for managing ***Unit*** objects later).

2. AI-Assisted Method Implementation:

- Focus on the ***enroll_student_in_unit*** method within the ***EnrollmentSystem***.
- Prompt:** Provide the ***EnrollmentSystem*** scaffold (or relevant parts) and the method signature to your AI assistant. Ask it to implement the method based on these steps:
 - Get the ***SchoolRegistry*** instance (e.g., ***self.registry***).
 - Use the registry to find the ***Student*** object by ***student_id***.

- iii) Find the **Unit** object by **unit_code** (Assume units are stored in a dictionary **self.units** within **EnrollmentSystem** for now - you might need to add this).
 - iv) If the student or unit is not found, raise an **InvalidDataError** (assume this custom exception exists). Ensure the student object retrieved is actually a **Student**.
 - v) (Optional: Add prerequisite check later if **check_prerequisites** function is ready).
 - vi) Call the found **unit** object's **add_student** method, passing the found **student** object.
 - vii) Wrap the logic in a **try...except** block to catch potential **InvalidDataError** (from step 4) and **EnrollmentError** (raised by **unit.add_student** if full/invalid). Print informative messages in the **except** blocks.
 - viii) If successful, print a success message.
- c) **Critically review the generated code.** Does it handle errors correctly? Does it access registry and units appropriately? Refine the code manually or with further prompts.
3. **AI-Assisted Debugging:**
- a) **Introduce a Bug:** Deliberately introduce a bug into your code (e.g., in **Student.display_info**, try accessing **self.__student_id** directly instead of **self.get_student_id()**). Run code that triggers the bug and copy the *full error message* (traceback).
 - b) **Prompt:** Paste the error message and the relevant code block (the method where the error occurs) into your AI assistant. Ask: "I'm getting this error: [paste error message]. Here is the code that causes it: [paste code block]. Explain why this error is happening in Python OOP and suggest how to fix it."
 - c) Evaluate the explanation and suggested fix. Apply the fix and verify.
 - d) **Try another:** Introduce a **TypeError** (e.g., passing incompatible types). Repeat the debugging process: run -> get error -> copy error & code -> prompt AI -> evaluate -> fix.
4. **Creative Challenge (Optional, if time permits):**
- a) **Try Top-Down Design with AI:** Start with a high-level goal: "Outline the main Python classes and methods needed for a system to generate a student's transcript." Ask the AI assistant. Then, use the assistant to help scaffold and implement each part based on the outline, starting from the highest level.